AI Video Editing Platform: Database Schema Blueprint

Version 0.1 (draft). Database: MongoDB 7 with Mongoose 8. Companion to the Functional & Technical Specification, which defines the Edit Plan JSON that this schema stores and resolves.

1. Purpose and design goals

Video editing data is deeply nested (project, timeline, tracks, clips, effects, keyframes, captions) and is read as a whole far more often than it is queried in pieces. The schema is built around that access pattern.

Design goals

- **Load an editor session in one read.** The current timeline, with all tracks and clips, is a single document.
- **Immutable history.** Every accepted edit produces a new snapshot, so undo, redo, compare and re-render of any version are cheap.
- **Separate intent from state.** The `editPlans` collection stores what the AI decided (the prompt and its JSON plan). The `timelines` collection stores the resolved, editable result in output time.
- **Stay under the 16 MB document limit** by moving unbounded data (caption cues, analysis, chat) into their own collections.
- **Safe concurrent edits** through an optimistic `rev` counter.

How Edit Plan relates to Timeline

```
prompt ─▶ editPlans (intent, seconds, source-time layers)
              |  resolve (cuts applied, source->output time, ids assigned)
              ▼
          timelines (clips and tracks, integer ms, output time)
              |  compile
              ▼
          FFmpeg render (renderJobs)
```

The resolver is deterministic: the same plan plus the same analysis always yields the same timeline. Manual edits (drag a clip, trim a caption) modify the timeline directly and are recorded as a new version with `source: "manual"`.

2. Conventions

Topic	Rule
Primary keys	`_id` as ObjectId. Public API IDs are prefixed strings stored in `publicId` (`prj_` , `ast_` , `trk_` , `clp_` , `cap_`)
Time	Integer **milliseconds** (`startMs` , `durMs` , `srcInMs`) snapped to the project frame grid. Avoids float drift. The Edit Plan uses seconds, converted at resolve time
Timestamps	`createdAt` , `updatedAt` (Mongoose `timestamps: true`), optional `deletedAt` for soft delete
Ownership	Every document carries `userId` and (where relevant) `projectId` so every query can be scoped and indexed by tenant
Enums	Lowercase strings, validated in Mongoose and `$jsonSchema`
Embedding rule	Embed data that is always read together and bounded (tracks, clips, effects, keyframes). Reference data that is unbounded, shared or large (assets, caption cues, analysis, jobs)
Colors	Hex strings. Positions normalized 0 to 1 so they are resolution independent

3. Collection map

Collection	Purpose	Size per doc	Relationship
`users`	Account, tier, quota	small	1 user to N projects
`projects`	Metadata, pointers, version index	small	1 project to 1 head timeline, N versions
`assets`	Uploaded and library media	small	N per project, referenced by clips
`analysisResults`	Transcript, silences, scenes, beats	medium to large	1 per asset

Collection	Purpose	Size per doc	Relationship
`timelines`	**Current** editable timeline	medium (50 to 500 KB)	1 per project
`timelineVersions`	Immutable snapshots	medium	N per project
`captionSets`	Subtitle cues with word timing	medium to large	referenced by subtitle clips
`editPlans`	AI plans and prompts	small	N per project
`chatMessages`	Prompt and response log	small	N per project
`renderJobs`	Preview and export jobs	small	N per project, points to a version
`catalog`	Templates, LUTs, fonts, music	small	referenced by clips and effects

```
users 1──N projects 1──1 timelines (head)
                  │ │        │
                  │ │        └─ refs: assets, captionSets, catalog
                  │ └─N timelineVersions (snapshots)
                  ├─N assets 1──1 analysisResults
                  ├─N editPlans, chatMessages
                  └─N renderJobs ──▶ timelineVersions.version
```

4. Project collection

The project is the entry point: metadata, ownership, status and a lightweight index of versions. It never contains timeline content.

Field	Type	Notes
`_id`, `publicId`	ObjectId, string	`prj_xxxxxxxx`
`userId`	ObjectId	Owner. Indexed
`title`	string	1 to 120 chars
`status`	enum	`draft`, `uploading`, `analyzing`, `ready`, `rendering`, `error`, `archived`

Field	Type	Notes
`settings`	object	`aspect` (16:9, 9:16, 1:1, 4:5), `fps` (24, 25, 30, 60), `width`, `height`, `language`, `brandKitId`
`baseAssetId`	ObjectId	Primary uploaded video
`timelineId`	ObjectId	Head timeline document
`headVersion`	int	Latest saved version number
`publishedVersion`	int or null	Version last exported
`versions`	array (capped at 50)	Lightweight index: `{version, label, source, createdAt, planId, thumbKey}`. Full data lives in `timelineVersions`
`stats`	object	`durationMs`, `assetCount`, `renderCount`, `storageBytes`
`thumbnailKey`	string	S3 key
`tags`	string[]	Optional
`createdAt`, `updatedAt`, `deletedAt`	date	

```
const ProjectSchema = new Schema({
  publicId: { type: String, required: true, unique: true },
  userId: { type: Schema.Types.ObjectId, ref: 'User', required: true, index:
true },
  title: { type: String, required: true, maxlength: 120 },
  status: { type: String, enum:
['draft','uploading','analyzing','ready','rendering','error','archived'],
default: 'draft' },
  settings: {
    aspect: { type: String, enum: ['16:9','9:16','1:1','4:5'], default:
'16:9' },
    fps: { type: Number, enum: [24,25,30,60], default: 30 },
    width: Number, height: Number, language: { type: String, default: 'en' },
    brandKitId: { type: Schema.Types.ObjectId, ref: 'Catalog' }
  },
  baseAssetId: { type: Schema.Types.ObjectId, ref: 'Asset' },
  timelineId: { type: Schema.Types.ObjectId, ref: 'Timeline' },
  headVersion: { type: Number, default: 0 },
  publishedVersion: { type: Number, default: null },
```

```
  versions: [{ _id: false, version: Number, label: String, source: String,
               planId: Schema.Types.ObjectId, thumbKey: String, createdAt:
Date }],
  stats: { durationMs: Number, assetCount: Number, renderCount: Number,
storageBytes: Number },
  deletedAt: Date
}, { timestamps: true });
```

Sample document

```
{
  "publicId": "prj_8f2a91c4",
  "userId": "66f1...a1",
  "title": "My Trip to Goa",
  "status": "ready",
  "settings": { "aspect": "16:9", "fps": 30, "width": 1920, "height": 1080,
"language": "en" },
  "baseAssetId": "66f1...b2",
  "timelineId": "66f1...c3",
  "headVersion": 7,
  "publishedVersion": 6,
  "versions": [
    { "version": 6, "label": "Cinematic + intro", "source": "ai", "planId":
"66f1...d4" },
    { "version": 7, "label": "Subtitles bigger", "source": "ai", "planId":
"66f1...d5" }
  ]
}
```

5. Timeline collection

One document per project holds the **head** (current) timeline. It is the structure the editor loads, renders on the canvas, and sends to the compiler.

5.1 Top-level fields

Field	Type	Notes
`projectId`, `userId`	ObjectId	Unique index on `projectId`
`rev`	int	Optimistic concurrency counter, incremented per write
`version`	int	Matches `projects.headVersion` and the snapshot in `timelineVersions`

Field	Type	Notes
`fps`, `width`, `height`	number	Copied from project settings at creation
`durationMs`	int	Derived: max track end. Recomputed on every write
`tracks`	Track[]	Ordered bottom to top (see 5.2)
`markers`	Marker[]	`{id, tMs, label, kind}` where kind is `user`, `scene`, `beat`
`globalEffects`	Effect[]	Whole-timeline effects such as color grade or audio normalize
`sourcePlanId`	ObjectId	Edit Plan that produced this state, null for manual edits

5.2 Track (embedded)

Field	Type	Notes
`id`	string	`trk_xxxx`, stable across versions
`kind`	enum	`video`, `audio`, `music`, `subtitle`, `text`, `overlay`
`order`	int	Z-order for visual tracks, lane order for audio
`name`	string	Display label
`locked`, `muted`, `hidden`	bool	Editor state that is saved with the timeline
`volumeDb`	number	Audio and music tracks only
`clips`	Clip[]	Sorted by `startMs` ascending

5.3 Clip (embedded)

All clips share a base, then add fields by `type`.

Common field	Type	Notes
`id`	string	`clp_xxxx`
`type`	enum	`video`, `audio`, `music`, `subtitle`, `text`, `overlay`, `intro`, `outro`
`startMs`, `durMs`	int	Position on the **output** timeline

Common field	Type	Notes
`enabled`	bool	
`effects`	Effect[]	Ordered chain applied to this clip
`keyframes`	object	Map of property to `[{tMs, value, ease}]`, with `tMs` relative to clip start
`transitionOut`	Transition or null	Applied between this clip and the next on the same track
`origin`	object	`{layerId, source: "ai"

Type	Extra fields
`video`	`assetId`, `srcInMs`, `srcOutMs`, `speed` (0.25 to 4), `transform {x,y,scale,rotation,opacity}`, `audioGainDb`, `reverse`
`audio`, `music`	`assetId` or `catalogId`, `srcInMs`, `srcOutMs`, `gainDb`, `fadeInMs`, `fadeOutMs`, `duck {enabled, thresholdDb, ratio}`, `beatSnap`
`subtitle`	`captionSetId`, `style {preset, font, size, color, highlightColor, position, animation, maxWordsPerLine}`
`text`	`content`, `style`, `position {x,y}`, `animation {in, out, durMs}`
`overlay`	`catalogId` or `assetId`, `transform`, `blendMode`
`intro`, `outro`	`templateId`, `params {text, subtext, theme}`

Effect

```
{ "id": "fx_1", "type": "color", "enabled": true,
  "params": { "preset": "cinematic", "contrast": 1.15, "saturation": 0.9,
"lut": "teal_orange" } }
```

`type` is one of `color`, `blur`, `vignette`, `grain`, `speed_ramp`, `denoise`, `normalize`, `stabilize`. `params` are validated per type, the same ranges as the Edit Plan.

Transition

```
{ "type": "dissolve", "durMs": 500, "easing": "ease_in_out" }
```

5.4 Invariants (enforced by the service layer)

1. Within one track, clips never overlap, **except** for the overlap equal to `transitionOut.durMs` with the next clip. The next clip starts at `prev.startMs + prev.durMs - transition.durMs`.
2. `srcOutMs - srcInMs` divided by `speed` equals `durMs` for video and audio clips.
3. `srcOutMs` does not exceed the referenced asset duration.
4. All times are multiples of one frame (`1000 / fps`, rounded) for video tracks.
5. Tracks of kind `video` need `assetId` clips only. `subtitle` tracks need `captionSetId` only.
6. At most 12 tracks and 2,000 clips per timeline. Subtitle cues do not count (they live in `captionSets`).
7. `durationMs` equals the greatest `startMs + durMs` across all tracks.

5.5 Sample timeline document

```
{
  "projectId": "66f1...a2",
  "rev": 31,
  "version": 7,
  "fps": 30, "width": 1920, "height": 1080,
  "durationMs": 58400,
  "tracks": [
    { "id": "trk_v1", "kind": "video", "order": 0, "name": "Main",
      "clips": [
        { "id": "clp_i1", "type": "intro", "startMs": 0, "durMs": 5000,
          "templateId": "tpl_cinematic", "params": { "text": "My Trip to Goa"
} },
        { "id": "clp_v1", "type": "video", "assetId": "66f1...b2",
          "srcInMs": 0, "srcOutMs": 4200, "startMs": 5000, "durMs": 4200,
"speed": 1,
          "transitionOut": { "type": "dissolve", "durMs": 500 },
          "effects": [ { "id": "fx_1", "type": "color", "params": { "preset":
"cinematic" } } ],
          "origin": { "layerId": "lyr_c1", "source": "ai" } },
        { "id": "clp_v2", "type": "video", "assetId": "66f1...b2",
          "srcInMs": 5900, "srcOutMs": 60000, "startMs": 8700, "durMs":
54100, "speed": 1 }
      ] },
    { "id": "trk_s1", "kind": "subtitle", "order": 1, "name": "Captions",
      "clips": [ { "id": "clp_s1", "type": "subtitle", "captionSetId":
"66f1...e5",
                   "startMs": 5000, "durMs": 53400,
```

```
                    "style": { "preset": "word_pop", "font": "Montserrat",
"size": 56, "position": "bottom" } } ] },
    { "id": "trk_m1", "kind": "music", "order": 2, "name": "Music",
      "volumeDb": -12,
      "clips": [ { "id": "clp_m1", "type": "music", "catalogId":
"mus_calm_01",
                    "srcInMs": 0, "srcOutMs": 58400, "startMs": 0, "durMs":
58400,
                    "fadeInMs": 1500, "fadeOutMs": 3000, "duck": { "enabled":
true, "ratio": 8 } } ] }
  ],
  "markers": [ { "id": "mk_1", "tMs": 12400, "kind": "scene", "label": "Beach
cut" } ]
}
```

5.6 Mongoose schema (abridged)

```
const EffectSchema = new Schema({
  id: String, type: { type: String, enum:
['color','blur','vignette','grain','speed_ramp','denoise','normalize','stabil
ize'] },
  enabled: { type: Boolean, default: true }, params: Schema.Types.Mixed
}, { _id: false });

const ClipSchema = new Schema({
  id: { type: String, required: true },
  type: { type: String, required: true, enum:
['video','audio','music','subtitle','text','overlay','intro','outro'] },
  startMs: { type: Number, required: true, min: 0 },
  durMs: { type: Number, required: true, min: 1 },
  enabled: { type: Boolean, default: true },
  assetId: Schema.Types.ObjectId, catalogId: String, captionSetId:
Schema.Types.ObjectId, templateId: String,
  srcInMs: Number, srcOutMs: Number, speed: { type: Number, default: 1, min:
0.25, max: 4 },
  transform: Schema.Types.Mixed, style: Schema.Types.Mixed, params:
Schema.Types.Mixed,
  effects: [EffectSchema], keyframes: Schema.Types.Mixed,
  transitionOut: { type: new Schema({ type: String, durMs: Number, easing:
String }, { _id: false }), default: null },
  origin: { layerId: String, source: String }
}, { _id: false });

const TrackSchema = new Schema({
  id: String, kind: { type: String, enum:
['video','audio','music','subtitle','text','overlay'] },
```

```
  order: Number, name: String, locked: Boolean, muted: Boolean, hidden:
Boolean, volumeDb: Number,
  clips: [ClipSchema]
}, { _id: false });

const TimelineSchema = new Schema({
  projectId: { type: Schema.Types.ObjectId, required: true, unique: true },
  userId: { type: Schema.Types.ObjectId, required: true, index: true },
  rev: { type: Number, default: 0 }, version: { type: Number, default: 0 },
  fps: Number, width: Number, height: Number, durationMs: Number,
  tracks: [TrackSchema], markers: [Schema.Types.Mixed], globalEffects:
[EffectSchema],
  sourcePlanId: Schema.Types.ObjectId
}, { timestamps: true, minimize: false });
```

Per-type fields live in `Mixed` objects (`transform`, `style`, `params`) and are validated by Zod in the service layer, which keeps the Mongoose schema stable as new effect types are added.

6. Caption sets (subtitle timestamps)

A subtitle clip references a caption set. Cues can run to thousands of entries with per-word timing, so they are stored outside the timeline.

Field	Type	Notes
`projectId`, `assetId`	ObjectId	Source asset the speech came from
`language`	string	BCP-47
`origin`	enum	`whisper`, `manual`, `imported`
`basedOn`	ObjectId	Parent set when edited (immutable, copy on write)
`cues`	Cue[]	Sorted, non-overlapping
`contentHash`	string	SHA-256 of cues, used to dedupe renders and cache the ASS file

```
{
  "language": "en", "origin": "whisper",
  "cues": [
    { "id": "cue_1", "startMs": 800, "endMs": 3400, "text": "We landed in Goa
at sunrise",
      "speaker": null,
      "words": [ { "w": "We", "s": 800, "e": 950, "conf": 0.98 },
                 { "w": "landed", "s": 950, "e": 1300, "conf": 0.97 } ] }
  ]
}
```

Cue times are in **source time** (relative to the asset). The resolver maps them to output time at render by walking the video track's `srcInMs` / `startMs` pairs, so cutting a clip automatically drops or trims the affected cues without rewriting the caption set. Editing cue text creates a new set (`basedOn` = old) so old versions keep their captions. If a set exceeds about 8,000 cues, split into `captionChunks` of 500 cues each referenced by the set.

7. Timeline versions (history)

Each saved change inserts an **immutable full snapshot**. Because ids are stable and heavy data is referenced, snapshots are small and restoring is a copy.

Field	Type	Notes
`projectId` , `userId`	ObjectId	
`version`	int	Unique with `projectId`
`parentVersion`	int or null	Branching on undo then new edit is allowed
`source`	enum	`ai` , `manual` , `restore` , `import`
`label`	string	Short summary, AI generated for prompts
`planId` , `promptId`	ObjectId	Present when source is `ai`
`snapshot`	object	Full timeline body (tracks, markers, globalEffects, fps, size, durationMs)
`diffSummary`	object	`{clipsAdded, clipsRemoved, effectsChanged}` for the history panel
`snapshotHash`	string	Detects no-op saves and lets render jobs reuse outputs

Field	Type	Notes
`createdAt`	date	

Write path (single transaction)

1. Read head timeline, check `rev` matches what the client sent.
2. Apply change, run invariants (5.4), recompute `durationMs`.
3. Insert `timelineVersions` snapshot with `version = headVersion + 1`.
4. Update `timelines` with `$set` of body plus `$inc: {rev: 1}` filtered on the old `rev`.
5. Update `projects` with `$set: {headVersion}` and `$push` on `versions` with `$slice: -50`.

A mismatched `rev` returns 409, and the client refetches and retries. **Undo** never deletes anything: it loads an older snapshot into the head with `source: "restore"`, which creates a new version. **Compaction:** after 90 days, intermediate `manual` snapshots may be thinned, keeping all `ai` versions and all versions referenced by `renderJobs`.

8. Edit plans and chat

`editPlans` stores the LLM output exactly as validated.

Field	Notes
`projectId`, `version`	Unique pair. Plan version equals the timeline version it produced
`parentVersion`	Plan it patched
`prompt`	Raw user text
`plan`	Full Edit Plan JSON (layers array)
`patch`	RFC 6902 patch when produced from a follow-up prompt
`model`, `tokensIn`, `tokensOut`, `latencyMs`	Cost and quality tracking
`assumptions`	Defaults the planner filled in
`validation`	`{repairAttempts, warnings[]}`

`chatMessages` keeps the conversation: `{projectId, role: "user"|"assistant"|"system", text, planId, versionResult, createdAt}`. It is separate from plans so chat can be paginated and trimmed without touching edit data.

9. Assets and analysis

`assets`

Field	Notes
`projectId`, `userId`, `kind`	`video`, `audio`, `image`, `font`, `lut`
`source`	`upload`, `library`, `generated`
`s3`	`{bucket, rawKey, proxyKey, audioKey, posterKey}`
`media`	`{durationMs, width, height, fps, videoCodec, audioCodec, sampleRate, channels, bitrate, sizeBytes, rotation}`
`status`	`uploading`, `processing`, `ready`, `failed`
`checksum`	SHA-256, to deduplicate uploads per user

`analysisResults` (one per asset, large arrays kept out of `assets`)

```
{
  "assetId": "66f1...b2", "version": 2, "status": "complete",
  "silences": [ { "startMs": 4200, "endMs": 5900, "avgDb": -52 } ],
  "sceneCuts": [ { "tMs": 12400, "score": 0.62 } ],
  "beats": { "bpm": 96, "grid": [ 420, 1045, 1670 ] },
  "loudness": { "integratedLufs": -21.3, "truePeakDb": -3.1 },
  "transcriptRef": "66f1...e5",
  "highlights": [ { "startMs": 30200, "endMs": 38000, "score": 0.91,
"reason": "key statement" } ]
}
```

10. Render jobs

Field	Notes
`projectId`, `userId`	

Field	Notes
`timelineVersion`, `snapshotHash`	Exactly what was rendered. Same hash plus same profile reuses the existing output
`type`	`analysis`, `preview`, `export`
`profile`	`preview_480p`, `final_1080p`, `final_4k`
`state`	`queued`, `active`, `completed`, `failed`, `canceled`
`percent`, `etaSec`	Updated at most once per second
`queueJobId`	BullMQ id
`output`	`{key, sizeBytes, durationMs, checksum}`
`error`	`{code, message, retryable}`
`timings`	`{queuedAt, startedAt, finishedAt, cpuSec}`
`expiresAt`	TTL date for previews

11. Catalog and users

`catalog`: `{kind: "template"|"lut"|"font"|"music"|"overlay", key, name, tags[], s3Key, meta, license: {name, attributionRequired, url}, active}`. Music entries add `bpm`, `mood`, `durationMs`, `beatGridKey`. Templates add `paramSchema` so the UI and validator know which parameters they accept.

`users`: `{email, name, authProvider, tier, quota: {storageBytes, renderMinutes, promptsPerDay}, usage: {...}, createdAt}`.

12. Indexes

Collection	Index	Purpose
`projects`	`{userId: 1, updatedAt: -1}` (partial: `deletedAt` absent)	Dashboard project list
`projects`	`{publicId: 1}` unique	API lookups
`timelines`	`{projectId: 1}` unique	Load editor
`timelineVersions`	`{projectId: 1, version: -1}` unique	History list, restore

Collection	Index	Purpose
`timelineVersions`	`{projectId: 1, snapshotHash: 1}`	Dedupe and render reuse
`assets`	`{projectId: 1, kind: 1}`, `{userId: 1, checksum: 1}`	Asset panel, upload dedupe
`analysisResults`	`{assetId: 1}` unique	Planner context
`captionSets`	`{projectId: 1, assetId: 1}`, `{contentHash: 1}`	Lookup and cache
`editPlans`	`{projectId: 1, version: -1}` unique	Plan history
`chatMessages`	`{projectId: 1, createdAt: 1}`	Pagination
`renderJobs`	`{projectId: 1, createdAt: -1}`, `{state: 1, type: 1}`	Job list, queue recovery
`renderJobs`	TTL on `expiresAt`	Preview cleanup
`catalog`	`{kind: 1, active: 1, tags: 1}`	Planner catalogs

13. Server-side validation example

Use `$jsonSchema` as a safety net behind Mongoose for the most important collection.

```
db.runCommand({
  collMod: 'timelines',
  validator: { $jsonSchema: {
    bsonType: 'object',
    required: ['projectId', 'userId', 'rev', 'version', 'fps', 'tracks'],
    properties: {
      rev: { bsonType: 'int', minimum: 0 },
      fps: { enum: [24, 25, 30, 60] },
      tracks: { bsonType: 'array', maxItems: 12,
        items: { bsonType: 'object', required: ['id', 'kind', 'clips'],
          properties: {
            kind: { enum:
['video','audio','music','subtitle','text','overlay'] },
            clips: { bsonType: 'array', maxItems: 2000 }
          } } }
    }
  } },
  validationLevel: 'moderate', validationAction: 'error'
});
```

Cross-clip invariants (overlap, source range) cannot be expressed in `$jsonSchema`, so they live in a shared `validateTimeline()` function used by the API, the resolver and a nightly integrity job.

14. Common queries

Need	Query
Dashboard list	`projects.find({userId, deletedAt: {$exists: false}}).sort({updatedAt: -1}).limit(20)`
Open editor	`timelines.findOne({projectId})` plus `assets.find({_id: {$in: usedAssetIds}})` plus `captionSets.find({_id: {$in: captionIds}})`
Update one clip	`timelines.updateOne({projectId, rev}, {$set: {'tracks.$[t].clips.$[c].startMs': v}, $inc: {rev: 1}}, {arrayFilters: [{'t.id': trackId}, {'c.id': clipId}]})`
History panel	`timelineVersions.find({projectId}, {snapshot: 0}).sort({version: -1})`
Planner context	`analysisResults.findOne({assetId})` plus current `timelines` head plus `catalog.find({active: true})`
Reuse render	`renderJobs.findOne({projectId, snapshotHash, profile, state: 'completed'})`

Project the `snapshot` field out of history list queries, since it is the heavy part. Stable `id` fields on tracks and clips are what make `arrayFilters` updates and patch paths work.

15. Scaling, retention and migration

- **Document size budget:** target under 500 KB per timeline, alarm at 4 MB. The 2,000 clip cap and externalized captions keep typical projects far below this.
- **Sharding (later):** shard `timelineVersions`, `renderJobs` and `chatMessages` by `{projectId: "hashed"}`. Keep `projects` and `timelines` unsharded until volume requires it.
- **Retention:** soft-deleted projects purged after 30 days (documents, S3 prefix, versions). Previews expire after 7 days (TTL). Failed uploads cleaned after 24 hours.

- **Schema versioning:** every timeline and snapshot carries `schemaVersion`. Migrations are idempotent scripts that upgrade head and, lazily on read, old snapshots.
- **Transactions:** required only for the version write path in Section 7 (multi-document). Use a replica set from day one, including local development.
- **Backups:** point-in-time recovery enabled, with weekly restore tests.
- **Privacy:** index and log by IDs only, never prompt text or transcript content in application logs.

16. Open questions

- Do you need multi-clip, multi-asset projects in the MVP? The schema supports them, but the Edit Plan currently has a single `baseAssetId`.
- Should collaboration (shared projects) be planned now? It would add `projects.members[]` with roles and a presence collection.
- Should snapshots be stored as full documents indefinitely, or moved to S3 as compressed JSON after 90 days?
- Brand kits (logos, fonts, colors) as a separate collection or part of `users`?